รายวิชา 31903-2011 การพัฒนาระบบจัดการฐานข้อมูล · ชิ้นงานสัปดาห์ที่ 3 (Scope Locked & Architecture)

ระบบบริหารจัดการหอพักแบบบูรณาการ (Dormitory Management System : หอพักสุขสบาย เพลส)

เอกสารข้อเสนอโครงการ ผังระบบงาน สถาปัตยกรรมซอฟต์แวร์ และการทวนสอบความต้องการ (อ้างอิงจาก Source Code ต้นแบบที่ใช้งานได้จริง เชื่อมต่อฐานข้อมูล PostgreSQL บน Supabase Singapore)

สรุปข้อมูลโครงการ (PROJECT BASELINE)

ขนาดระบบเป้าหมาย	หอพัก 20–40 ห้อง (1 อาคาร) • สกุลเงินบาท (THB) • ภาษาไทย
สถาปัตยกรรมหลัก	Next.js 16.4.3 + React 19.3 + TypeScript + Tailwind CSS v4
ระบบฐานข้อมูล	PostgreSQL (Supabase AWS ap-southeast-1) + Prisma ORM 6.19.3
ขอบเขตฐานข้อมูลจริง	**9 ตารางหลัก • 29 ความต้องการฟังก์ชัน • 36 เส้นทางเว็บไซต์ (Routes)**

คณะผู้จัดทำ (ทีมนักศึกษา 4 คน)

1. นายวชิรวิทย์ นวลคำ	Project Lead / Backend
2. นายพยุรรักษ์ ศิริรัช	Database / Business Logic
3. นายณัฐดนัย วงศ์ใจมา	Frontend / UI & UX
4. นายสิทธิศักดิ์ชญ์ นาคะเสน	QA / Testing & Documentation

ปัญหาของระบบเดิม (As-Is) สู่ระบบใหม่ (To-Be) และวัตถุประสงค์

งานที่ 1: เอกสารข้อเสนอโครงการ (Project Proposal) — เปลี่ยนการจดมือและไฟล์กระจัดกระจายเป็นฐานข้อมูลกลาง

1. ลดเวลาออกบิลรายเดือน

≤ 10 วินาที

ออกใบแจ้งหนี้พร้อมกัน 40 ห้อง (Batch Generation) จากเดิมใช้เวลาคำนวณมือ 2–3 ชั่วโมง

2. ความแม่นยำการคำนวณเงิน

ตรง 100%

ใช้ชนิดข้อมูล `Decimal(10,2)` ปัดเศษแบบ Half-up ตามกฎ BR-05, BR-09, BR-10 (NFR-08)

3. ความเร็วตอบสนองหน้าเว็บ

≤ 3 วินาที

แสดงผลรายการและรายงานที่ P95 เมื่อมีข้อมูลห้องพัก 40 ห้อง และบิลสะสม 1,500 ใบ (NFR-01)

4. ความง่ายในการใช้งานจริง

≤ 5 นาที

ผู้ใช้ใหม่เรียนรู้ไม่เกิน 10 นาที สามารถจดมิเตอร์และออกบิล 1 ห้องสำเร็จภายใน 5 นาที (NFR-07)

ตารางเปรียบเทียบกระบวนการทำงาน (As-Is vs To-Be)

มิติการทำงาน	ระบบเดิม (As-Is : สมุด/Excel)	ระบบใหม่ (To-Be : Web + PostgreSQL)
1. ข้อมูลผู้เช่า/สัญญา	เก็บกระดาษ/ไฟล์แยก ค้นหายาก เสี่ยงทำสัญญาซ้อนห้องเดียวกัน	เก็บในตาราง `tenants`, `contracts` บังคับ 1 ห้อง 1 สัญญา active (BR-02)
2. จดมิเตอร์น้ำ-ไฟ	จดใส่กระดาษแล้วพิมพ์ซ้ำลง Excel เสี่ยงกรอกเลขย้อนหลังผิด	กรอกผ่านตาราง `meter_readings` ดึงเลขเดือนก่อนอัตโนมัติ ห้ามน้อยกว่าเดิม (BR-04)
3. การออกบิล/รับชำระ	คำนวณผิดพลาดง่าย สลิปโอนเงินปะปนในแชท ตามหนี้ค้างยาก	ผู้เช่าแนบสลิปผ่านเว็บ แอดมินกดอนุมัติ ออกใบเสร็จ RCP อัตโนมัติ (BR-07, BR-08)
4. แจ้งซ่อมบำรุง	แจ้งทางโทรศัพท์หรือพูดปากเปล่า ลืมงาน ไม่มีประวัติรายห้อง	บันทึกใน `repair_requests` ติดตามสถานะ (Open → In Progress → Done)

กลุ่มผู้ใช้งานระบบ 3 บทบาท (Actors & RBAC)

บทบาท (Role)	หน้าที่และความต้องการหลักในระบบ	การควบคุมสิทธิ์
1. เจ้าของหอพัก (Admin)	จัดการผู้ใช้/บทบาท, ตั้งค่าเรทค่าน้ำ-ไฟกลาง, ยกเลิกบิล/ใบเสร็จ (Void), แก้มิเตอร์ตั้งต้น, ดูรายงานการเงิน	เข้าถึงได้ทุกเมนู รวมถึง `/admin/users`
2. เจ้าหน้าที่ (Staff)	บันทึกห้องพัก ผู้เช่า สัญญาเช่า จดมิเตอร์ ออกบิล ตรวจสลิปและรับชำระเงิน อัปเดตงานซ่อม	เข้าถึงโมดูลปฏิบัติการ (จำกัดสิทธิ์ลบ/Void)
3. ผู้เช่า (Tenant)	ดูบิลรายเดือนของตนเอง, **แนบสลิปโอนเงินผ่านมือถือ/PC**, แจ้งซ่อมและติดตามสถานะงานซ่อม	เฉพาะพอร์ทัลผู้เช่า (`/me/*`) ล็อกอินด้วยห้อง+เบอร์+เลขบัตร 4 ตัวท้าย

ขอบเขตโครงการ (In/Out of Scope) และโมดูลฟังก์ชัน A–J (29 ข้อ)

เชื่อมโยงความต้องการเชิงฟังก์ชัน (Must 22 ข้อ / Should 7 ข้อ) เข้ากับตารางฐานข้อมูลจริง 9 ตาราง

สรุป 10 โมดูลในขอบเขต (In-Scope) เชื่อมตารางฐานข้อมูลและหน้าจอ

โมดูล	รหัส SRD (Must / Should)	ตารางฐานข้อมูลที่ใช้	หน้าจอหลัก
A. ผู้ใช้และสิทธิ์	2.1, 2.2 Must + ตั้งค่ากลาง	`profiles`, `system_settings`	P01, P25
B. ห้องพัก	2.3, 2.4 Must (ผังห้องเรียลไทม์)	`rooms`, `contracts`	P02, P03, P04, P05
C. ผู้เช่า	2.5–2.9 Must (ตรวจบัตร 13 หลัก)	`tenants`	P06, P07, P08
D. สัญญาเช่า	2.10, 2.12 Must / 2.11, 2.13 Should	`contracts`, `rooms`	P09, P10, P11
E. มิเตอร์น้ำ-ไฟ	2.14, 2.15 Must (ตรวจเลขไม่ลดลง)	`meter_readings`	P12
F. ใบแจ้งหนี้	2.16–2.18 Must (ออกบิล/พิมพ์ PDF)	`invoices`, `meter_readings`	P13, P14, P15
G. การชำระเงิน	2.19, 2.20 Must (+ตรวจสลิปโอนเงิน)	`payments`, `invoices`	P15, P16, P17
H. งานซ่อมบำรุง	2.21, 2.22 Must / 2.23 Should	`repair_requests`	P18, P19, P20
I. พอร์ทัลผู้เช่า	2.24, 2.25 Should (+แนบสลิปโอนเงิน)	`invoices`, `repair_requests`	P26, P27, P28, P29
J. รายงานสรุป	2.26, 2.28 Must / 2.27, 2.29 Should	ทุกตารางธุรกรรม	P21, P22, P23, P24

สิ่งที่อยู่นอกขอบเขต (Out-of-Scope) พร้อมเหตุผล

- **ตัดชำระเงินออนไลน์/เช็กยอดธนาคารอัตโนมัติ (Payment Gateway API):** มีค่าธรรมเนียมและต้องจดทะเบียนนิติบุคคล จึงใช้ *"ระบบผู้เช่าแนบสลิปให้แอดมินกดอนุมัติ"* แทน
- **แจ้งเตือนผ่าน LINE OA / Email อัตโนมัติ:** ลดการพึ่งพาโควตา API ภายนอก ให้ดูสถานะผ่านเว็บพอร์ทัลโดยตรง
- **อัตราค่าน้ำ-ไฟแบบขั้นบันได & คิดค่าเช่ารายวัน (Pro-rate):** หอพักขนาด 20–40 ห้อง ใช้อัตราคงที่ต่อหน่วยและคิดเต็มเดือน (OI-07, OI-08)
- **ระบบบัญชีแยกประเภท/ยื่นภาษี & แอปมือถือ Native & หลายอาคาร:** เกินขอบเขตเวลา 15 สัปดาห์ ใช้ Responsive Web (ตั้งแต่ 360px) แทน

จุดเด่นที่พัฒนาเพิ่มใน Source Code จริง (Value-Added Features)

- **ระบบแนบสลิปโอนเงิน (Tenant Slip Upload):** ผู้เช่าอัปโหลดรูปสลิปจากมือถือ/PC บันทึกในฟิลด์ `slip_url`, `slip_status` ให้แอดมินตรวจสอบและกดอนุมัติออกใบเสร็จได้ในคลิกเดียว
- **ตารางตั้งค่ากลาง (`system_settings`) & โปรไฟล์ (`profiles`):** ขยายจาก 7 ตาราง เป็น 9 ตาราง เพื่อรองรับการตั้งค่าเรทค่าน้ำ-ไฟและเลขบัญชีพร้อมเพย์จากหน้าเว็บโดยไม่ต้องแก้โค้ด (แก้ประเด็น OI-01)
- **UI/UX มาตรฐานสูง:** รองรับสลับธีม สว่าง/มืด (Light/Dark Swap), แถบเมนูย่อ-ขยายได้, ระบบแจ้งเตือน Toast Popup, และระบบป้องกันการกรอก Emoji 100%

กฎธุรกิจหลัก (BR-01 ถึง BR-14) และข้อกำหนดที่ไม่ใช่ฟังก์ชัน (NFR)

บังคับใช้จริงทั้งระดับ Frontend Validation, Server Action และ Database Constraints

สรุปกฎธุรกิจหลัก 14 ข้อ (Business Rules)

รหัส	รายละเอียดการบังคับใช้ในระบบและฐานข้อมูล
BR-01, 13, 14	**ข้อมูลผู้เช่า & PDPA:** บัตร ปชช. 13 หลักตรวจสูตร Mod 11 Checksum · หน้าตารางปิดบังเป็น `•••••••••0121` (BR-13) · เบอร์โทร 10 หลักขึ้นต้นด้วย 0 ตัดขีดอัตโนมัติ (BR-14)
BR-02, 03	**สัญญาเช่า & สถานะห้อง:** 1 ห้องมีสัญญา `active` ได้ไม่เกิน 1 ฉบับ (Partial Unique Index) · เปิดสัญญา → ห้องเป็น `occupied` · สิ้นสุดสัญญา → ห้องเป็น `vacant` ใน Transaction เดียวกัน
BR-04	**เลขมิเตอร์น้ำ-ไฟ:** จำนวนเต็มไม่ติดลบ ค่าปัจจุบัน ≥ ค่าก่อนหน้า · Unique `(room_id, reading_month)` · กรณีเปลี่ยนมิเตอร์ Admin แก้เลขตั้งต้นได้พร้อมหมายเหตุ
BR-05, 06	**สูตรคำนวณบิล:** ยอดรวม = ค่าเช่า + ปัด2ตำแหน่ง(หน่วยน้ำ×เรทน้ำ) + ปัด2ตำแหน่ง(หน่วยไฟ×เรทไฟ) + ค่าอื่น + ค่าปรับ ชนิด `Decimal(10,2)` · Unique `(contract_id, billing_month)`
BR-07, 08, 09	**สถานะบิล & การรับชำระ:** คำนวณจากผลรวมยอดชำระที่ `valid` (0=unpaid, บางส่วน=partial, ครบ=paid) · ห้ามลบใบเสร็จ ใช้การ `voided` พร้อมเหตุผล · ค่าปรับคิดได้ครั้งเดียวเมื่อเลย `due_date`
BR-10	**การคืนเงินมัดจำ:** `A = max(0, D − X)`, หักยอดค้าง `min(A, O)` ด้วยวิธีชำระ `deposit_offset`, เงินคืนผู้เช่า = `A − หักยอดค้าง`
BR-11, 12	**เลขเอกสาร & Audit Trail:** บิล `INV-YYYYMM-ห้อง`, ใบเสร็จ `RCP-YYYYMM-XXXX` · ห้ามลบข้อมูลธุรกรรม (Soft Status: `ended`, `cancelled`, `voided`)

ข้อกำหนดที่ไม่ใช่ฟังก์ชัน (NFR-01 ถึง NFR-10)

ความปลอดภัยและข้อมูลส่วนบุคคล (NFR-03, NFR-04)

Must

- ตรวจสอบสิทธิ์และบทบาท (Session & Role) ที่ฝั่งเซิร์ฟเวอร์ทุก Action
- คิวรีผ่าน Prisma ORM แบบ Parameterized ป้องกัน SQL Injection 100%
- ปฏิบัติตาม พ.ร.บ. PDPA: ไม่เก็บไฟล์ภาพบัตรประชาชน ปิดบังเลขบัตรในหน้ารายการ และไม่ส่งเลขบัตรผ่าน URL หรือ Log

ความจุ ประสิทธิภาพ และการสำรองข้อมูล (NFR-01, 02, 05, 06)

Must

- **ความจุ (NFR-02):** ข้อมูลข้อความ 40 ห้อง นาน 3 ปี ใช้พื้นที่ฐานข้อมูลไม่เกิน 50 MB (อยู่ในโควตา 500 MB ของ Supabase Free)
- **การสำรองข้อมูล (NFR-06):** รันคำสั่ง `pg_dump` สัปดาห์ละ 1 ครั้ง และก่อนปิดรอบบิลรายเดือน เก็บไฟล์แยกนอก Git Repository
- **ความพร้อมใช้งาน (NFR-05):** ปลุกโปรเจกต์ Supabase ล่วงหน้าก่อนนำเสนอทุกครั้ง

คุณภาพโค้ดและการแสดงผล (NFR-07, 08, 09, 10)

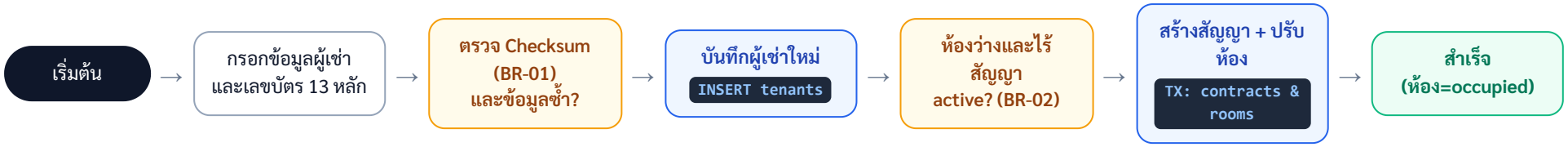
- **TypeScript Strict Mode:** ผ่านการตรวจ `tsc --noEmit` และ `next build` 0 Error
- **Responsive & Thai Date:** รองรับหน้าจอมือถือ 360px ขึ้นไป แสดงวันที่รูปแบบ พ.ศ. ไทย และเก็บในฐานข้อมูลเป็น ISO Date มาตรฐาน

ผังงานหลัก 3 กระบวนการ (Core Process Flowcharts)

งานที่ 2 (ส่วน A): แสดงการไหลของข้อมูลจากหน้าเว็บ (Input) ผ่านการตรวจกฎธุรกิจ (BR) ลงสู่ฐานข้อมูล (Storage)

ผังงานที่ 1: การเพิ่มผู้เช่าและทำสัญญาเช่าห้องพัก (อ้างอิง BR-01, BR-02, BR-03)

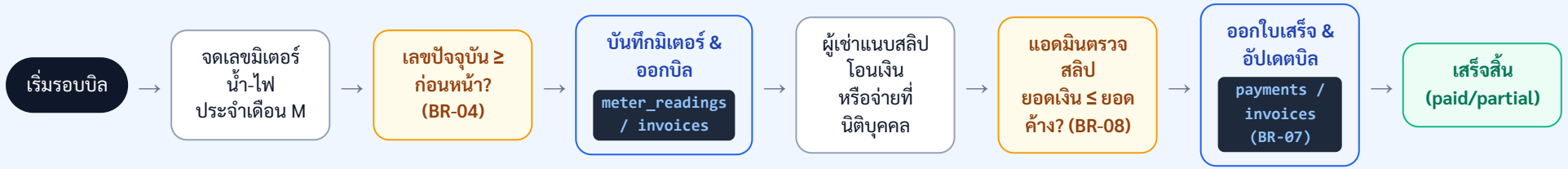
ตาราง: tenants, contracts, rooms



คำอธิบาย: เจ้าหน้าที่กรอกข้อมูลผู้เช่า ระบบตรวจสูตรเลขบัตร 13 หลัก (BR-01) หากไม่ผ่านให้แจ้งแก้ไข เมื่อผ่านจึงบันทึกใน `tenants` จากนั้นเลือกห้องว่าง ระบบตรวจ Partial Unique Index (BR-02) และรัน Database Transaction บันทึก `contracts` พร้อมเปลี่ยนสถานะ `rooms` เป็น `occupied` (BR-03)

ผังงานที่ 2 [ภาพหลักสำหรับนำเสนอ]: รอบบิลรายเดือน จดมิเตอร์ ออกบิล แนบสลิป และรับชำระ (BR-04 ถึง BR-09)

ตาราง: meter_readings, invoices, payments



เหตุผลที่เลือกเป็นภาพหลักในการนำเสนอ: เพราะแสดงหัวใจของระบบฐานข้อมูลหอพักครบวงจร ตั้งแต่การป้องกันจดมิเตอร์ผิด (BR-04) การคำนวณเงินด้วย Decimal (BR-05) การเชื่อมโยงผู้เช่าแนบสลิปผ่านเว็บ ไปจนถึงการออกใบเสร็จและอัปเดตสถานะบิลอัตโนมัติ (BR-07, BR-08)

ผังงานที่ 3: การแจ้งซ่อมบำรุงและติดตามสถานะงานซ่อม (อ้างอิง ความต้องการ 2.21–2.25)

ตาราง: repair_requests



แผนผังเว็บไซต์ (Site Map P01–P29) และความสัมพันธ์กับ 9 ตารางฐานข้อมูล

งานที่ 2 (ส่วน B): ออกแบบตามกฎ 3-Click Rule แยกสิทธิ์ชัดเจน พร้อมตรวจความครบถ้วน 100%

1. กลุ่มบริหารห้องพัก ผู้เช่า และสัญญา (Admin/Staff)

- **P01 เข้าสู่ระบบ** (`/login`) → คัดกรองบทบาท (2.1)
- **P02 แดชบอร์ด & ผังห้อง** (`/dashboard`) → ภาพรวม (2.4, 2.11)
- **P03–P05 ห้องพัก** (`/rooms`, `/new`, `/[id]`) → ตาราง `rooms` (2.3, 2.23)
- **P06–P08 ผู้เช่า** (`/tenants`, `/new`, `/[id]`) → ตาราง `tenants` (2.5–2.9)
- **P09–P11 สัญญาเช่า** (`/contracts`, `/new`, `/[id]`) → ตาราง `contracts` (2.10–2.13)

2. กลุ่มการเงิน งานซ่อม และรายงาน (Admin/Staff)

- **P12 จดมิเตอร์รายเดือน** (`/meters`) → `meter_readings` (2.14, 2.15)
- **P13–P15 ใบแจ้งหนี้** (`/invoices`, `/generate`, `/[id]`) → `invoices` (2.16–2.18)
- **P16–P17 การชำระเงิน & ใบเสร็จ** (`/payments`, `/receipt`) → `payments` (2.19, 2.20)
- **P18–P20 งานแจ้งซ่อม** (`/repairs`, `/new`, `/[id]`) → `repair_requests` (2.21–2.23)
- **P21–P24 รายงาน 4 ฉบับ** (`/reports/*`) & **P25 ผู้ใช้** (`/admin/users`) (2.2, 2.26–2.29)

3. กลุ่มพอร์ทัลผู้เช่า (Tenant Portal) & Wireframe

- **P26 หน้าแรกผู้เช่า** (`/me`) → สรุปสัญญา ยอดบิล และบัญชีโอนเงิน (2.24)
- **P27 บิลและแนบสลิปของฉัน** (`/me/invoices`) → ดูบิล + อัปโหลดสลิป (2.24)
- **P28–P29 แจ้งซ่อมผู้เช่า** (`/me/repairs`, `/new`) → แจ้งและติดตามซ่อม (2.25)

3 หน้าที่แนะนำทำ UI Wireframe ก่อนในสัปดาห์ถัดไป:
1) **P02 แดชบอร์ดผังห้อง** (ศูนย์รวมข้อมูลทุกตาราง)
2) **P12 บันทึกมิเตอร์รายเดือน** (ตารางกรอกข้อมูลหลายห้องพร้อมกัน)
3) **P15 รายละเอียดบิล/ตรวจสลิป/รับชำระ** (จุดตัดยอดเงินสำคัญที่สุด)

ตารางตรวจสอบความครบถ้วน: 9 ตารางฐานข้อมูลจริง ↔ หน้าเว็บ (P01–P29) ↔ ความต้องการ (2.1–2.29)

ลำดับ	ชื่อตารางในฐานข้อมูล (PostgreSQL)	บทบาทหน้าที่หลักและข้อจำกัด (Constraints)	หน้าเว็บที่อ่าน/เขียนข้อมูล	ข้อกำหนด SRD ที่รองรับ	ผลตรวจ
1–2	`profiles`, `system_settings`	เก็บผู้ใช้ 3 บทบาท (RBAC) และเรทค่าน้ำ/ไฟ/ค่าปรับ/บัญชีธนาคารกลาง	P01, P10, P25, P26	ข้อ 2.1, 2.2 (แก้ประเด็น OI-01)	ครบ 100%
3–4	`rooms`, `tenants`	ข้อมูลห้องพัก (Unique `room_no`) และผู้เช่า (Unique `id_card_no` 13 หลัก)	P02–P08	ข้อ 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9	ครบ 100%
5	`contracts`	เชื่อมห้องกับผู้เช่า (M:N), Partial Unique Index 1 สัญญา active ต่อห้อง	P02, P05, P08, P09–P11	ข้อ 2.10, 2.11, 2.12, 2.13	ครบ 100%
6–7	`meter_readings`, `invoices`	เลขมิเตอร์รายเดือน และใบแจ้งหนี้พร้อมสถานะสลิปโอนเงิน (`slip_status`)	P12, P13–P15, P21, P23, P27	ข้อ 2.14–2.18, 2.24, 2.26, 2.28	ครบ 100%
8–9	`payments`, `repair_requests`	ใบเสร็จรับเงิน (ห้ามลบ ใช้ `voided`) และใบแจ้งซ่อมรายห้อง	P15–P20, P24, P28–P29	ข้อ 2.19–2.23, 2.25, 2.29	ครบ 100%

การเลือกใช้สถาปัตยกรรม (Tech Stack Selection) และตารางตัดสินใจ

งานที่ 3: สถาปัตยกรรม 3 ชั้น (3-Tier Architecture) ที่สอดคล้องกับกฎธุรกิจและข้อจำกัดแผนฟรี

1. ชั้นแสดงผล (Web UI Layer)

Tailwind CSS v4 + React 19.3 Client/Server Components

- เทียบกับ Bootstrap: Tailwind CSS ปรับแต่ง Responsive (360px+) และทำระบบสลับธีม **Light/Dark Mode** ได้คล่องตัวกว่า ไม่ติดหน้าตาสำเร็จรูป
- มี Client Validation ดักเลขบัตร 13 หลัก และบล็อก Emoji ทันทีขณะพิมพ์

2. ชั้นประมวลผล (Server / Logic Layer)

Next.js 16.4.3 (App Router) + TypeScript + Prisma ORM

- *หมายเหตุสำคัญ:* React เป็นเพียง UI Library ส่วน **Next.js** คือ Full-stack Framework ที่รวมทั้งฝั่งหน้าเว็บและ Server Actions ไว้ในโปรเจกต์เดียว
- ลดภาระการดูแล 2 โค้ดเบส ใช้ภาษา TypeScript เดียวกันตั้งแต่หน้าจอถึงฐานข้อมูล

3. ชั้นฐานข้อมูล (Database Layer)

PostgreSQL บน Supabase (AWS Singapore Region)

- เทียบกับ MySQL: **PostgreSQL** รองรับ **Partial Unique Index** (`WHERE status = 'active'`) ตามกฎ BR-02 ได้โดยตรง
- รองรับ ACID Transaction (BR-03) และชนิด `NUMERIC/Decimal` แม่นยำสูง (BR-05)

ตารางตัดสินใจแบบถ่วงน้ำหนัก (Weighted Decision Matrix) [สมมติฐานน้ำหนักโดยทีม]

เกณฑ์การประเมิน (น้ำหนัก)	Next.js + PostgreSQL	Laravel + MySQL	Django + PostgreSQL
1. ความเข้ากันกับกฎธุรกิจ BR-02/BR-05 (25%)	**9.5 / 10** (รองรับ Partial Index)	7.5 / 10 (MySQL ไม่มี Partial Index)	9.0 / 10 (รองรับครบ)
2. ความเร็วในการพัฒนาภายใน 15 สัปดาห์ (20%)	**9.0 / 10** (Type-safe ทั้งระบบ)	8.5 / 10 (ต้องแยกภาษา PHP/JS)	8.0 / 10 (ต้องแยก Python/JS)
3. ความถนัดของทีมและเอกสารศึกษา (20%)	**9.0 / 10** (ทีมคุ้นเคย TS/React)	7.5 / 10	7.0 / 10
4. ต้นทุนโฮสติ้งฟรีเพื่อการศึกษา (15%)	**9.5 / 10** (Vercel + Supabase Free)	6.5 / 10 (หาโฮสต์ PHP ฟรียาก)	7.0 / 10
5. ประสิทธิภาพและการ Deploy อัตโนมัติ (10%)	**9.5 / 10** (CI/CD ผ่าน GitHub)	7.5 / 10	7.5 / 10
6. ความปลอดภัยและ ORM Type Safety (10%)	**9.5 / 10** (Prisma + Strict TS)	8.5 / 10 (Eloquent ORM)	8.5 / 10 (Django ORM)

มาตรการรับมือข้อจำกัดแผนฟรีและความเสี่ยง (Risk Mitigation)

ข้อจำกัด / ความเสี่ยง	แผนรับมือที่ปฏิบัติจริงในโครงการ
Supabase Free พักโปรเจกต์เมื่อไร้การใช้งาน 7 วัน	ตั้งตารางเข้าใช้งานทุกสัปดาห์ และมี Checklist กดกู้คืน/ตรวจสอบสถานะฐานข้อมูลล่วงหน้า 24 ชม. ก่อนวันนำเสนอทุกครั้ง (NFR-05)
โควตาฐานข้อมูล 500 MB & ไม่มี Auto Backup	ข้อมูล 40 ห้อง 3 ปีใช้ไม่เกิน 50 MB; สั่งสำรองข้อมูลด้วย pg_dump ทุกสัปดาห์เก็บในเครื่องส่วนตัวของทีม (NFR-02, NFR-06)
Vercel Hobby จำกัดเฉพาะการใช้พาณิชย์ (Non-Commercial)	ใช้เพื่อการศึกษาในรายวิชาเท่านั้น หากเจ้าของหอพักนำไปใช้งานจริงเชิงพาณิชย์ มีเอกสารคู่มือย้ายขึ้น VPS หรือ Vercel Pro ทันที (OI-12)
เวลาเขียนโค้ดมีจำกัด (สัปดาห์ 7–10 รวม ~80 ชม.)	จัดลำดับพัฒนา **Core MVP (Must 22 ข้อ)** ให้เสร็จสมบูรณ์ก่อน แล้วจึงต่อยอดข้อ Should 7 ข้อ (ปัจจุบันทีมทำต้นแบบครบทั้ง 29 ข้อแล้ว)

แผนการนำเสนอ (Pitching), ตอบคำถามกรรมการ และการทวนสอบ 5 เสาหลัก

งานที่ 4 และ งานที่ 5: แบ่งบทพูดทีม 4 คน เก็งคำถามป้องกันโครงการ และผลตรวจ Requirement Verification

ผู้พูด 1: วชิรวิทย์ (1.5 นาที)

เปิดด้วย Storytelling: เล่าเหตุการณ์สิ้นเดือนที่เจ้าของหอต้องเปิดสมุดจดมิเตอร์เทียบ Excel หลายไฟล์ คิดบิลผิดและสลิปตกหล่น แล้วสรุปขอบเขต In/Out-of-Scope

ผู้พูด 2: พชรรักษ์ (1.5 นาที)

เจาะลึก Flowchart & กฎธุรกิจ: อธิบายผังงานรอบบิลรายเดือน การป้องกันเลขมิเตอร์ถอยหลัง (BR-04) การคำนวณเงิน Decimal (BR-05) และการแนบสลิป

ผู้พูด 3: ณัฐดนัย (1.0 นาที)

อธิบาย Site Map & UI/UX: แสดงโครงสร้าง 29 หน้าใน 3 คลิก แยกพอร์ทัลผู้เช่ากับแอดมิน พร้อมสาธิตหน้าเว็บจริงที่สลับธีม สว่าง/มืด ได้

ผู้พูด 4: สัณห์พิชญ์ (1.0 นาที)

Tech Stack & ความเป็นไปได้: สรุปเหตุผลที่เลือก Next.js + PostgreSQL (Supabase Singapore), แผนสำรองข้อมูล `pg_dump` และขออนุมัติล็อกขอบเขต

ชุดคำตอบคำถามคณะกรรมการ (Defend the Feasibility)

คำถามสำคัญจากกรรมการ	คำตอบสั้นกระชับพร้อมหลักฐานอ้างอิง
1. ทำไมไม่เชื่อมระบบตัดเงินอัตโนมัติ แต่ใช้วิธีแนบสลิปแทน?	เพื่อควบคุมขอบเขตให้เสร็จใน 15 สัปดาห์และไม่ต้องเสียค่าธรรมเนียม Payment Gateway โดยผู้เช่าแนบสลิปเข้าตาราง `invoices` แล้วให้ Staff กดอนุมัติสร้างใบเสร็จใน `payments` (BR-07, BR-08)
2. ทำไมเลือก PostgreSQL แทน MySQL?	เพราะกฎ **BR-02** กำหนดให้ 1 ห้องมีสัญญา `active` ได้เพียง 1 สัญญา ซึ่ง PostgreSQL รองรับการสร้าง *Partial Unique Index* (`WHERE status = 'active'`) ในระดับฐานข้อมูลโดยตรง
3. ข้อมูลเลขบัตรประชาชนปลอดภัยตาม PDPA อย่างไร?	ระบบเก็บเฉพาะตัวเลข 13 หลักที่ผ่าน Checksum (BR-01) ไม่เก็บไฟล์รูปบัตร ปิดบัง 9 หลักแรกเป็น `•••••••••0121` ในหน้ารายการ (BR-13) และตรวจสิทธิ์ฝั่งเซิร์ฟเวอร์ทุกครั้ง (NFR-03, NFR-04)
4. หากทำไม่ทันในสัปดาห์ที่ 7–10 จะตัดอะไรก่อน?	ยึดแผน MVP ตัวเลือก A คือส่งมอบข้อ **Must 22 ข้อ** (ห้อง-ผู้เช่า-สัญญา-มิเตอร์-บิล-รับชำระ) ให้สมบูรณ์ 100% ก่อน ส่วนข้อ Should 7 ข้อจัดเป็นส่วนขยาย (ปัจจุบันทำต้นแบบครบแล้ว)

ผลการทวนสอบความต้องการด้วย 5 เสาหลัก (5-Pillar Verification)

เสาหลักการทวนสอบ	ผลตรวจ	หลักฐานยืนยันจากเอกสารและซอร์สโค้ดจริง
1. Validity (แก้ปัญหาตรงจุด)	PASS	ทุกฟังก์ชันตอบโจทย์ปัญหาบิลล่าช้า คิดเลขผิด และตามหนี้ยากของหอพัก 20–40 ห้องโดยตรง
2. Consistency (ไม่ขัดแย้งกัน)	PASS	รหัส 2.1–2.29, กฎ BR-01–BR-14 และ NFR-01–NFR-10 สอดคล้องตรงกันทั้งเอกสารและโค้ด
3. Completeness (ครบถ้วนสมบูรณ์)	PASS	ครบทั้ง 9 ตารางฐานข้อมูล มีหน้าเว็บ P01–P29 รองรับทุกตาราง ไม่มีตารางลอยหรือหน้าเว็บว่าง
4. Feasibility (ทำจริงได้ทันเวลา)	PASS	มี Working Prototype ที่ผ่าน `next build` ทั้ง 36 Routes และเชื่อม Supabase จริงแล้ว
5. Verifiability (ตรวจสอบวัดผลได้)	PASS	ไร้คำกำกวม ทุกข้อมีเกณฑ์วัดผลชัดเจน (เช่น ≤ 3 วินาที, ปัดเศษ 2 ตำแหน่งตรง 100%)

ประเด็นที่ต้องยืนยันกับอาจารย์ที่ปรึกษา (Open Issues): (1) ยืนยันขอบเขตพอร์ทัลผู้เช่า [OI-02] (2) ยืนยันเกณฑ์ล็อกอินผิด 5 ครั้ง [OI-03] (3) ยืนยันการไม่ต้องส่ง DFD เนื่องจากใช้ Flowchart + Site Map ตามสไลด์สัปดาห์ที่ 2 [OI-04] (4) ขออนุญาตใช้ AI IDE ช่วยตรวจสอบโค้ดภายใต้การทำ Code Review ของทีม